Annexure-I

EMPLOYEE MANAGEMENT SYSTEM

Skill Development Project Report

BACHELOR OF TECHNOLOGY

(COMPUTER SCIENCE AND ENGINEERING)

Submitted to

LOVELY PROFESSIONAL UNIVERSITY

PHAGWARA, PUNJAB

From 10/06/25 to 19/07/25

Course code :CSE443

Submitted by

Anjana V

Reg.No: 12322576

Under the Guidance of

Dr. Sarneet Kaur

Annexure-II: Student Declaration

To whom so ever it may concern

I, **Anjana V, 12322576,** hereby declare that the work done by me on "**Library Management System**" is a record of original work for the partial fulfillment of the requirements for the award of the degree, **B.Tech (Computer Science and Engineering).**

TRAINING CERTIFICATE

CENTRE FOR
PROFESSIONAL ENHANCEMENT

Certificate No. 405699

NAAC GRADE A++

Certificate of Merit

This is to certify that Mr./Ms. **Anjana V** S/D/W/o **Mr. Sudheesh Kumar V**

student of **School of Computer Science and Engineering** Registration No. **12322576**

pursuing **Bachelor of Technology (Computer Science and Engineering)** completed

skill development course named **Advanced Data Structure**

organized by **Centre for Professional Enhancement** Lovely Professional University

from **10 June 2025** to **19 July 2025** and obtained **C** Grade.

Date of Issue : 13-08-2025
Place of Issue: Phagwara (India)

Prepared by
(Administrative Officer-Records)

Programme Coordinator
Centre for Professional Enhancement

Head of School
School of Computer Applications

ACKNOWLEDGEMENT

I would like to express my sincere gratitude for providing me with the opportunity to undergo summer training in **Advanced Data Structure**. I extend my heartfelt thanks to my mentors and guides who have been a constant source of knowledge and motivation throughout the project duration. I am also thankful to my friends and family for their support and encouragement during this journey.

CHAPTER 1: INTRODUCTION

Company Profile:

This project was developed as part of an academic summer training program under the guidance of the School of Computer Science and Engineering. Though not associated with an external company, the approach and deliverables mimic a real-world software project.

Overview of Training Domain:

The training focused on system development using C++ and basic web technologies. The domain emphasized how to use data structures (like hash maps, queues, vectors) and object-oriented programming to build a functional system. The frontend aspect introduced the basics of HTML and CSS to make the application more user-friendly.

Objective of the Project:

The goal of the project was to create a **Library Management System** that simplifies operations such as adding books, borrowing/returning books, and tracking popular titles and borrowers. The project also aimed to provide an understanding of STL (Standard Template Library) and how it can be used effectively in real-life applications.

CHAPTER 2: TRAINING OVERVIEW

Tools & Technologies Used:

- **C++** with STL for backend logic
- **Visual Studio Code** as the main code editor
- **HTML & CSS** for creating the frontend interface
- **Command Prompt / Terminal** for running the backend
- **Web browser** to display the frontend layout **Areas Covered During**

Training:

- Understanding and applying C++ STL containers (unordered map, map, vector, queue, priority queue)
- Designing functions to simulate real-world actions (e.g., borrowing a book)
- Creating basic user interfaces using HTML and CSS
- Implementing FIFO logic using queues

- Implementing max heap to track most borrowed books
- Maintaining real-time borrow records **Daily/Weekly Work Summary:**

- **Week 1**: Studied STL basics and project planning
- **Week 2**: Coded core modules like add/search/borrow book
- **Week 3**: Added borrower tracking, popular books, and return logic
- **Week 4**: Developed frontend (HTML/CSS) and connected it logically
- **Week 5**: Final testing, bug fixes, and report documentation

---

CHAPTER 3: PROJECT DETAILS

Title of the Project:

Library Management System using C++ and Web UI **Problem**

Definition:

Manually handling library data is inefficient and prone to errors. This project aims to provide a digital solution to manage books, borrowers, and track popular titles efficiently.

Scope and Objectives:

- Add/search books and authors
- Borrow and return books
- Track which books are borrowed and by whom
- Maintain a FIFO return queue
- Generate statistics (most popular books, total borrowers, etc.)
- Simple interface for users to interact with the system

System Requirements:

Software:

- C++ Compiler (G++ or MSVC)
- Web Browser
- Visual Studio Code

Hardware:

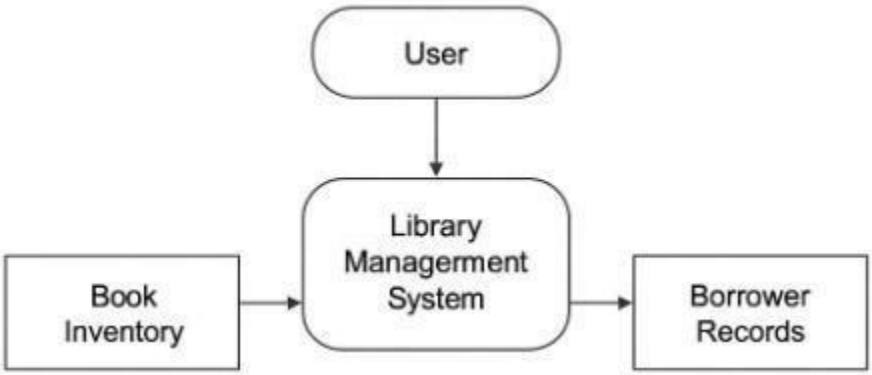
- 2 GB RAM or higher
- Any modern processor
- Basic monitor and keyboard **Architecture Diagram:**

The project is divided into two main parts:

1. **Backend in C++** – handles book logic and data
2. **Frontend in HTML/CSS** – simulates the user interface **Data Flow /**

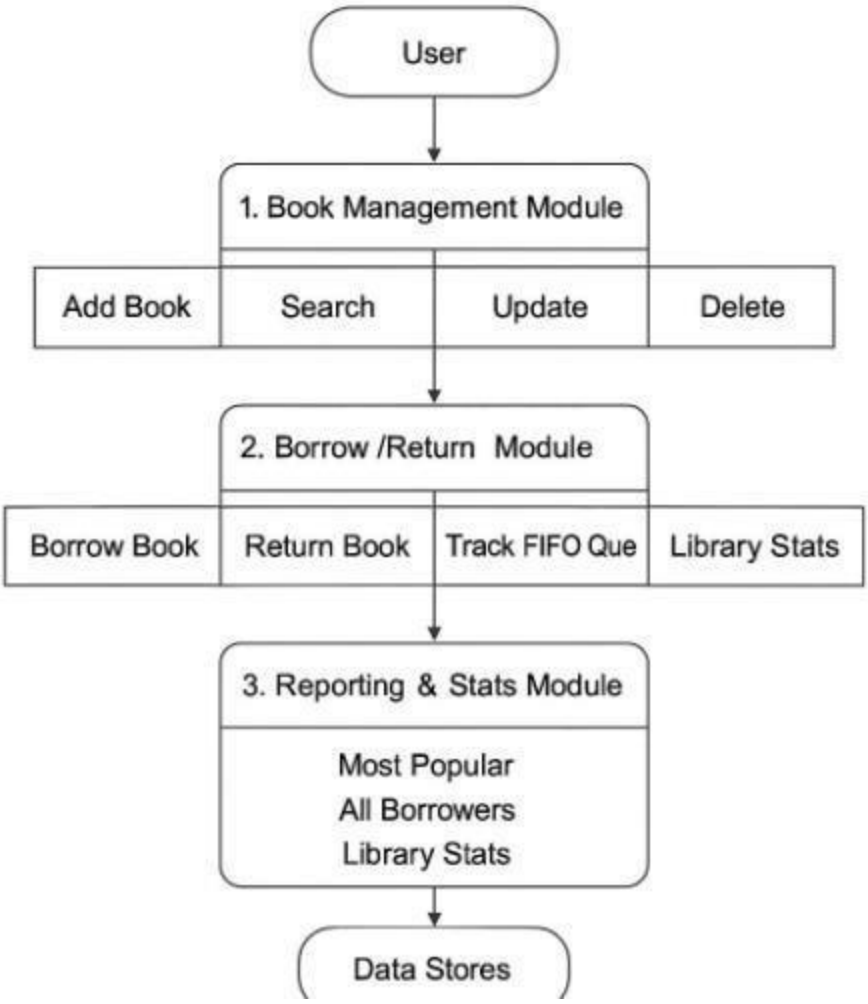
UML Diagrams:

1. Level 0 DFD – Context Level



- The user interacts with the Library Management System to perform action.
- The system reads/writes data from/to the Book Inventory and Borrower record

2. Level 1 DFD – Functional Decomposition



CHAPTER 4: IMPLEMENTATION

Tools Used:

- C++ compiler (GCC / MSVC)
- Visual Studio Code
- Web browser (Chrome, Firefox) **Methodology:**

The system is designed in a modular way. Each part—like adding a book or borrowing—was coded separately. Once each function worked properly, everything was combined into one working system. UI was added later to improve usability.

Modules / Screenshots:

1. **Add Book Module** – Input ID, title, author, and number of copies

```
void addBook(string id, string title, string author, int copies = 1) {
    if (books.count(id)) {
        cout << "❌ Book with ID '" << id << "' already exists!\n";
        return;
    }

    books[id] = Book(id, title, author, copies);
    bstByTitle[title] = &books[id];

    cout << "✅ Book added successfully!\n";
    cout << "    📖 Title: " << title << "\n";
    cout << "    👤 Author: " << author << "\n";
    cout << "    🆔 ID: " << id << "\n";
    cout << "    📚 Copies: " << copies << "\n";
}
```

2. **Search Module** – Search using ID and view book status and borrowers

```
void searchBook(string id) {
    if (books.count(id)) {
        Book& book = books[id];
        cout << "\n📖 Book Found:\n";
        printDivider();
        cout << "    🆔 ID: " << book.id << "\n";
        cout << "    📚 Title: " << book.title << "\n";
        cout << "    👤 Author: " << book.author << "\n";
        cout << "    📊 Borrow Count: " << book.borrowCount << " times\n";
        cout << "    📚 Available: " << (book.totalCopies - book.borrowedCopies) << "/" << book.totalCopies << "\n";
        cout << "    📍 Status: " << (book.borrowedCopies >= book.totalCopies ? "🔴 All Copies Borrowed" : "🟢 Available") << "\n";

        if (!book.borrowers.empty()) {
            cout << "    👥 Current Borrowers:\n";
            for (auto &borrower : book.borrowers) {
                cout << "      - " << borrower.second << " (ID: " << borrower.first << ")\n";
            }
        }
        printDivider();
    } else {
        cout << "❌ Book with ID '" << id << "' not found!\n";
    }
}
```

3. **Borrow Module** – Input borrower name, ID, and phone number

```
void borrowBook(string bookId, string borrowerId, string borrowerName, string phone) {
    if (!books.count(bookId)) {
        cout << "❌ Book with ID '" << bookId << "' not found!\n";
        return;
    }

    Book& book = books[bookId];
    if (book.borrowedCopies >= book.totalCopies) {
        cout << "❌ No copies available for borrowing!\n";
        return;
    }

    // Add or update borrower
    borrowers[borrowerId] = Borrower(borrowerId, borrowerName, phone);
    borrowers[borrowerId].borrowedBooks.push_back(bookId);

    // Update book
    book.borrowedCopies++;
    book.borrowCount++;
    book.borrowers.push_back({borrowerId, borrowerName});
    borrowQueue.push(borrowerId);
    maxHeap.push({book.borrowCount, bookId});

    cout << "✅ Book borrowed successfully!\n";
    cout << "   📖 '" << book.title << "' by " << book.author << "\n";
    cout << "   👤 Borrower: " << borrowerName << " (ID: " << borrowerId << ")\n";
    cout << "   📞 Phone: " << phone << "\n";
    cout << "   📅 Due date: 14 days from now\n";
}
```

4. **Return Module** – Return using borrower ID or FIFO

```
void returnBookByBorrowerId(string borrowerId) {
    bool found = false;

    for (auto &pair : books) {
        Book& book = pair.second;
        auto it = find_if(book.borrowers.begin(), book.borrowers.end(),
                          [&borrowerId](const pair<string, string>& p) {
                              return p.first == borrowerId;
                          });

        if (it != book.borrowers.end()) {
            string borrowerName = it->second;
            book.borrowers.erase(it);
            book.borrowedCopies--;

            // Remove from borrower's list
            if (borrowers.count(borrowerId)) {
                auto& borrower = borrowers[borrowerId];
                borrower.borrowedBooks.erase(
                    remove(borrower.borrowedBooks.begin(), borrower.borrowedBooks.end(), book.id),
                    borrower.borrowedBooks.end()
                );

                // Remove borrower if no more books
                if (borrower.borrowedBooks.empty()) {
                    borrowers.erase(borrowerId);
                }
            }

            cout << "🔄 Book returned successfully!\n";
            cout << "   📖 '" << book.title << "' by " << book.author << "\n";
            cout << "   👤 Returned by: " << borrowerName << " (ID: " << borrowerId << ")\n";
            cout << "   ✅ Thank you for returning on time!\n";
            found = true;
            break;
        }
    }

    if (!found) {
        cout << "❌ Borrower ID '" << borrowerId << "' not found!\n";
    }
}
```

```
void returnBook() {
    if (!borrowQueue.empty()) {
        string borrowerId = borrowQueue.front();
        borrowQueue.pop();
        returnBookByBorrowerId(borrowerId);
    } else {
        cout << "📚 No books are currently borrowed.\n";
    }
}
```

5. **Popular Book Module** – Uses a heap to find the most borrowed book

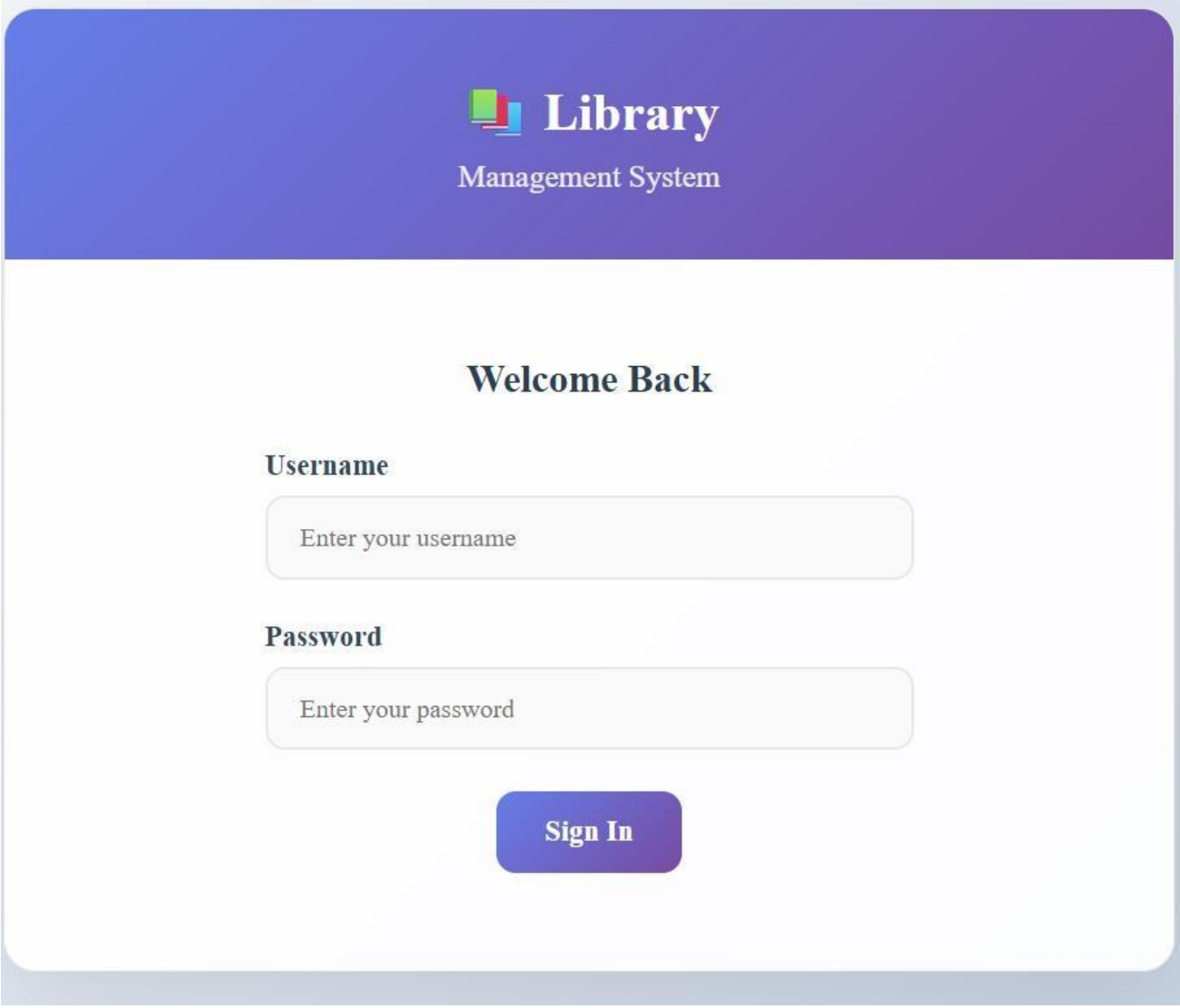
```
void mostPopular() {
    if (maxHeap.empty()) {
        cout << "📊 No borrowing data available.\n";
        return;
    }

    string id = maxHeap.top().second;
    Book& book = books[id];

    cout << "\n🔥 Most Popular Book:\n";
    printDivider();
    cout << "   📖 Title: " << book.title << "\n";
    cout << "   👤 Author: " << book.author << "\n";
    cout << "   📊 Borrowed: " << book.borrowCount << " times\n";
    cout << "   📚 Available: " << (book.totalCopies - book.borrowedCopies) << "/" << book.totalCopies << "\n";
    cout << "   📍 Status: " << (book.borrowedCopies >= book.totalCopies ? "🔴 All Copies Borrowed" : "🟢 Available") << "\n";
    printDivider();
}
```

6. **Borrowers Module** – Lists all borrowers and their current books

```cpp
void showBorrowersList() {
    if (borrowers.empty()) {
        cout << "📭 No active borrowers found.\n";
        return;
    }

    cout << "\n👥 Active Borrowers List:\n";
    printDivider();

    int totalBorrowedBooks = 0;
    for (auto &borrower : borrowers) {
        totalBorrowedBooks += borrower.second.borrowedBooks.size();
    }

    cout << "📊 Summary:\n";
    cout << "   👥 Total Active Borrowers: " << borrowers.size() << "\n";
    cout << "   📚 Total Borrowed Books: " << totalBorrowedBooks << "\n\n";

    for (auto &pair : borrowers) {
        Borrower& borrower = pair.second;
        cout << "👤 " << borrower.name << "\n";
        cout << "   📞 Phone: " << borrower.phone << "\n";
        cout << "   🆔 ID: " << borrower.id << "\n";
        cout << "   📚 Books Borrowed: " << borrower.borrowedBooks.size() << "\n";

        if (!borrower.borrowedBooks.empty()) {
            cout << "   📖 Borrowed Books:\n";
            for (int i = 0; i < borrower.borrowedBooks.size(); i++) {
                string bookId = borrower.borrowedBooks[i];
                if (books.count(bookId)) {
                    Book& book = books[bookId];
                    cout << "      " << (i+1) << ". " << book.title << " by " << book.author << " (ID: " << bookId << ")\n";
                }
            }
        }
        cout << "\n";
    }
    printDivider();
}
```

7. **Frontend (Web)** – Simple HTML/CSS layout simulating operations

Website Screenshots:


Library
Management System
Welcome Back
Username
Enter your username
Password
Enter your password
Sign In

Library

Management System

Welcome to the Library

Choose an option to get started

Borrow Book
Borrow books from library

Return Book
Return borrowed books

Add Book
Add new books to library

Search Book
Search for specific books

All Books
View all library books

Statistics
View library statistics

Borrowers
View active borrowers

Popular Books
Most borrowed books

Library

Management System

Borrow Book

Book ID

Enter book ID

Borrower Name

Enter borrower name

Borrower ID

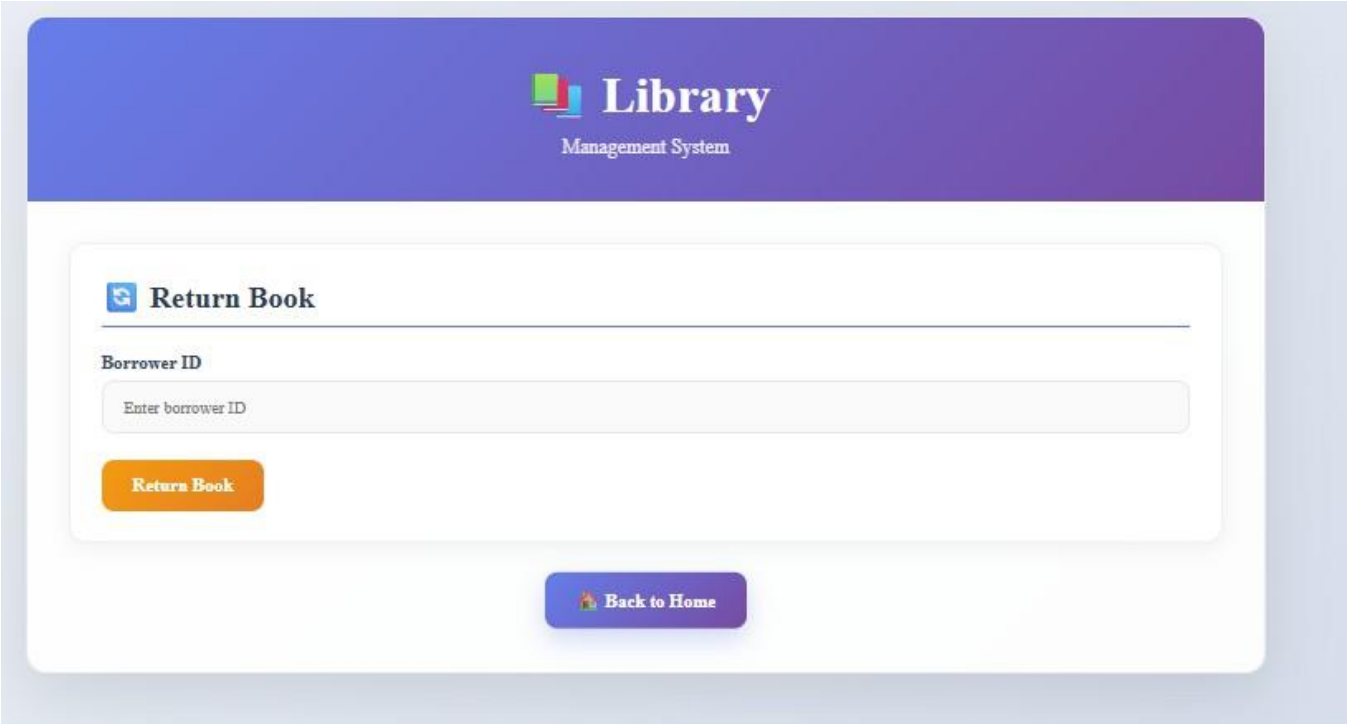
Enter borrower ID

Phone Number

Enter phone number

Borrow Book

Back to Home


Library
Management System
Return Book
Borrower ID
Enter borrower ID
Return Book
Back to Home

Library

Management System

Add New Book

Book ID

Enter unique book ID (e.g., B1301)

Book Title

Enter book title

Author

Enter author name

Number of Copies

1

Add Book

Back to Home

Library

Management System

All Books

The Great Gatsby

by F. Scott Fitzgerald

ID: B101 | Borrowed: 15 times | Available: 4/5 | Status: Available

To Kill a Mockingbird

by Harper Lee

ID: B102 | Borrowed: 12 times | Available: 3/3 | Status: Available

Pride and Prejudice

by Jane Austen

ID: B103 | Borrowed: 18 times | Available: 4/4 | Status: Available

1984

by George Orwell

ID: B104 | Borrowed: 22 times | Available: 6/6 | Status: Available

The Catcher in the Rye

by J.D. Salinger

ID: B105 | Borrowed: 8 times | Available: 2/2 | Status: Available

The Alchemist

by Paulo Coelho

ID: B201 | Borrowed: 25 times | Available: 4/4 | Status: Available

The Kite Runner

by Khaled Hosseini

ID: B202 | Borrowed: 14 times | Available: 3/3 | Status: Available

Library
Management System

Library Statistics

11 Total Books	1 Books Borrowed	1 Active Borrowers
228 Total Borrows		

Back to Home

Library
Management System

Active Borrowers

John Doe
Phone: 123-456-7890
ID: BR001
Books Borrowed: 1

Back to Home

Popular Books

11	228	20.7
Total Books	Total Borrows	Avg Borrows/Book

45
Top Book Borrows

1
Harry Potter and the Sorcerer's Stone
by J.K. Rowling
ID: B901 | Borrowed: 45 times | Available: 8/8 | Status: Available

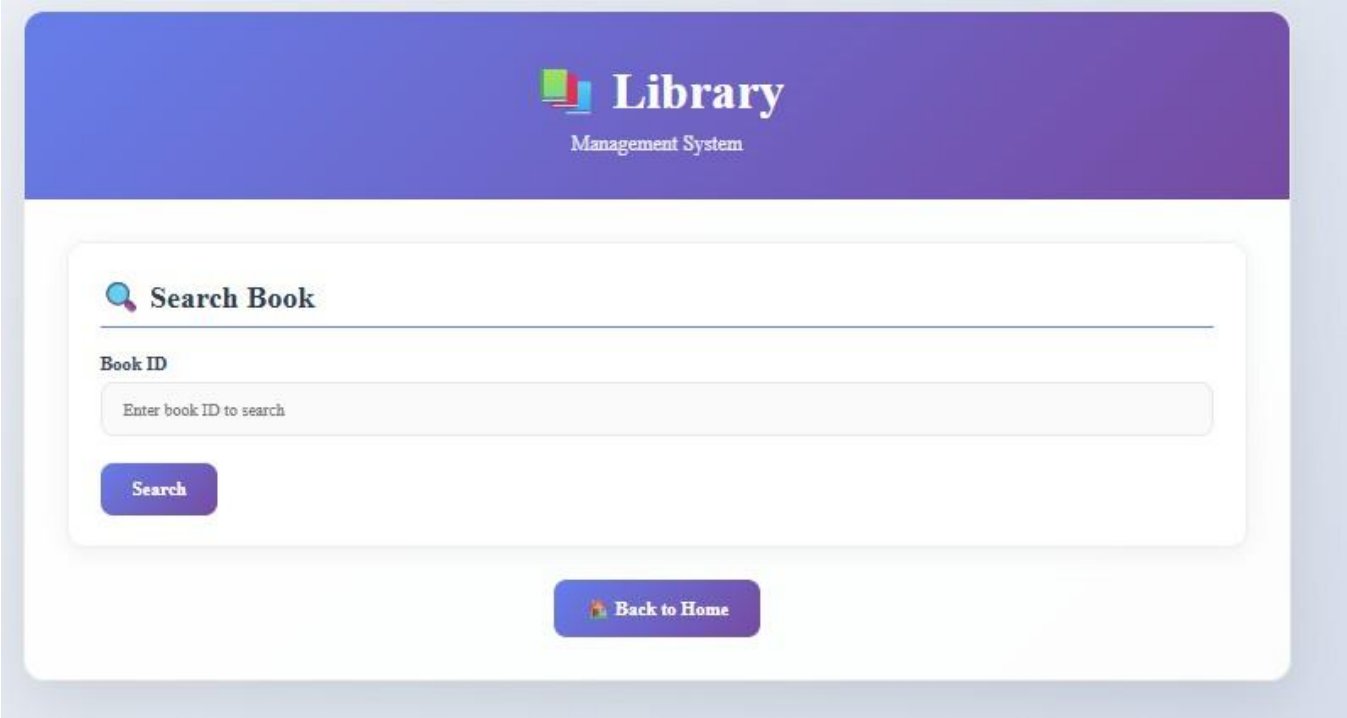
2
The Da Vinci Code
by Dan Brown
ID: B401 | Borrowed: 32 times | Available: 7/7 | Status: Available

3
Sapiens: A Brief History of Humankind
by Yuval Noah Harari
ID: B501 | Borrowed: 26 times | Available: 4/4 | Status: Available

4
The Alchemist
by Paulo Coelho
ID: B201 | Borrowed: 25 times | Available: 4/4 | Status: Available

5
1984
by George Orwell
ID: B104 | Borrowed: 22 times | Available: 6/6 | Status: Available

6
Pride and Prejudice



CHAPTER 5: RESULTS AND DISCUSSION

Output / Report:

- The system successfully allows books to be added, borrowed, returned, and searched.
- Stats like most popular books and total borrowers work as expected.
- Borrower history is maintained until the book is returned.
- The frontend allows basic interaction using buttons and fields.

Challenges Faced:

- Managing user input and data consistency in C++
- Ensuring all edge cases (e.g., duplicate IDs, no available copies) are handled
- Styling frontend to look decent and be responsive
- Tracking book borrow count accurately

Learnings:

- Hands-on experience with STL containers
- Learned how to manage and link frontend with backend
- Got confidence in writing modular, functional C++ code
- Understood how real systems simulate operations like borrowing, tracking, and reporting

CHAPTER 6: CONCLUSION

This Library Management System project was a practical way to apply programming knowledge to solve a real-world problem. From writing the logic in C++ using STL to designing a simple web interface, the project helped me understand how different components work together. I also learned to think modularly and debug effectively. Overall, this training experience helped me grow in both logic-building and problem-solving skills, making me more confident in software development.